🎓 Guía Paso a Paso: Descifrando URDF, `robot_description` y TF2 en Burger Delivery

> **Importante**
>
> **Actualización del Repositorio Privado del Equipo:** Antes de iniciar o continuar con este taller, asegúrese de haber sincronizado su repositorio privado de equipo con los últimos cambios de la base del curso. Consulte la Guía Oficial de Sincronización y Actualizaciones para realizar este proceso correctamente.

> **Nota**
>
> **El modelo vendorizado es el del robot del laboratorio: Gen3 de 6 GDL, pinza Robotiq 2F-85 y módulo de visión en la muñeca** (re-vendorizado desde `kortex_description/arms/gen3/6dof` con `python3 scripts/revendorizar_gen3_6dof.py --vision <ros2_kortex>/kortex_description`). El mismo script regenera la copia del visor web, `visual/burger_delivery_gen3.urdf`.
>
> Conviene no confundir dos modelos que coexisten en el workspace:
>
Modelo	Cadena cinemática	GDL
`kortex_description`, generado en vivo por el driver	`base_link → shoulder_link → bicep_link → forearm_link → …`	6
`burger_description` (vendorizado)	`gen3_base_link → gen3_shoulder_link → gen3_bicep_link → gen3_forearm_link → …`	6
>
> Tienen la misma cinemática, pero el de `burger_description` lleva todos sus frames bajo el prefijo `gen3_` y lo publica `display.launch.py`, un visor sin robot. El RViz que abre `kinova_connection.launch.py` con `launch_rviz:=true` usa el del driver, que refleja la posición articular real. Los URDF de referencia de Kinova en `vendor/robots/` siguen siendo los originales de 7 GDL.

> **Precaución**
>
> **Este taller no usa el driver del robot: no aplica la convención de estación anfitriona, y `display.launch.py` no debe compartir dominio con una sesión del robot ni con los visores de otras estaciones.** El launch publica sin namespace `/joint_states` (desde `joint_state_publisher_gui`), `/tf`, `/tf_static` y `/robot_description`: los mismos tópicos que el driver real. En el dominio `0` durante una sesión con el robot, tus deslizadores se mezclarían con la telemetría del brazo, que es el síntoma de "dos drivers en el mismo dominio" de `TROUBLESHOOTING.md` §2.1; y con varias estaciones practicando a la vez, los modelos saltarían entre los deslizadores de todos. Trabaja este taller en el dominio de tu equipo (§2.4 y §4.1), en **todas** las terminales, y vuelve al `0` del curso al terminar:
>
> ```
> export ROS_DOMAIN_ID=<11, 12, ... el de tu equipo>
> timeout 5s ros2 daemon stop; ros2 daemon start
> ```
>
> Aísla el dominio; no cambies `ROS_AUTOMATIC_DISCOVERY_RANGE` a `LOCALHOST`.

¡Bienvenido! En esta guía vamos a explorar juntos el corazón de la descripción robótica de nuestro proyecto. El objetivo es que aprendas a leer, validar y ajustar la estructura de `burger_delivery` sin tratar el URDF como un simple "XML decorativo".

🧠 Contexto: ¿Qué es realmente un URDF?

Antes de tocar el código, es vital entender qué estamos haciendo. **URDF** (*Unified Robot Description Format*) es, literalmente, el **ADN de tu robot**.

En robótica, no basta con tener un modelo 3D bonito. El ordenador necesita un **modelo matemático** para responder preguntas críticas:

- ¿Hacia dónde puede girar este brazo?
- ¿Si muevo el motor 10 grados, dónde quedará la pinza?
- ¿Esta pieza chocará con la mesa si avanzo?

El URDF traduce la realidad física a un lenguaje que ROS 2 entiende, dividiendo al robot en dos elementos básicos:

1. **Links (Eslabones):** Son las partes rígidas (los "huesos"). Tienen peso, apariencia y volumen.
2. **Joints (Articulaciones):** Son las uniones que permiten el movimiento (las "bisagras"). Definen cómo se conecta un link con otro.

Esta descripción es un **estándar en la industria**. Lo mejor de usar URDF es la **portabilidad**: el mismo archivo que usas hoy para ver el robot en RViz, lo usará MoveIt 2 para calcular rutas sin choques, y Gazebo o Ignition para simular la gravedad y los motores.

Sin un URDF bien configurado, el robot es solo un montón de metal ciego. Con él, se convierte en una entidad capaz de interactuar con su entorno con precisión milimétrica.

Cada sección de esta guía combina tres elementos clave:

1. **El Concepto:** ¿Qué estamos tratando de entender?
2. **La Realidad:** Revisión real en los archivos del proyecto.
3. **La Acción:** Ejercicios prácticos y verificables en terminal o RViz.

Nos enfocaremos en:

- La escena fija (`delivery_scene_fixed.urdf`).
- El manipulador Kinova Gen3.
- Los carritos modelados como URDFs separados.
- La conexión entre URDF, `robot_description`, `robot_state_publisher` y TF2.
- La localización por AprilTags usando `tag_mesa -> tag_carrito*`.
- La cámara de muñeca: cómo llevar un punto del marco óptico a la base del brazo, que es lo que necesita el Laboratorio IA 01 de Embodied AI (§11).

💡 **Un consejo antes de empezar:** Tómate tu tiempo. No conviene avanzar de sección si no se cumple su criterio de éxito. Cada paso confirma un concepto que usaremos más adelante. ¡Vamos allá!

0. ¡Manos a la obra! Preparación y Primera Prueba

🧠 El Concepto

ROS 2 no carga archivos nuevos automáticamente desde `src`. Los archivos que usa `ros2 launch` se resuelven desde el paquete instalado en `install/share`. Si agregas, renombras o editas URDFs, es **obligatorio** reconstruir el paquete.

🔍 Revisión en el Proyecto

El launch principal está en: `burger_description/launch/display.launch.py`. Este archivo es el encargado de cargar:

- `burger_description/urdf/delivery_scene_fixed.urdf` (Escena y Kinova).
- `burger_description/urdf/car1_apriltag.urdf` (Carrito 1).
- `burger_description/urdf/car2_apriltag.urdf` (Carrito 2).

🛠️ Tu primer reto: Construir y Validar

Vamos a preparar tu entorno. Ejecuta estos comandos en tu terminal:

1. **Comprueba el enlace de los recursos (Meshes):** El URDF busca los modelos en `package://burger_description/meshes/...`, pero en nuestro proyecto están dentro de `visual/meshes`. El puente es un enlace simbólico que **ya viene versionado en el repositorio**; sólo verifícalo:

   ```
   cd ~/ros2_ws/src/burger_delivery/burger_description
   ls -l meshes
   # Esperado: meshes -> visual/meshes
   ```

   Sólo si no existe (por ejemplo, porque tu sistema de archivos no conservó el enlace), créalo con `ln -s visual/meshes meshes`.

   > **Advertencia:** No vuelvas a ejecutar `ln -s visual/meshes meshes` si el enlace ya existe: como `meshes` apunta a un directorio, `ln` no falla sino que crea **otro** enlace roto *dentro* de él (`visual/meshes/meshes`). Desde el `CMakeLists.txt` se instala `visual/meshes` como `meshes`, así que la compilación no depende del enlace.

2. **Compila y refresca tu espacio de trabajo:**

   ```
   cd ~/ros2_ws
   colcon build --packages-select burger_description --symlink-install
   source install/setup.bash
   ```

3. **Verifica la instalación real:**

   ```
   ls -l install/burger_description/share/burger_description/urdf
   ls -l install/burger_description/share/burger_description/meshes
   ```

   Deben aparecer los archivos `.urdf` y los archivos `.stl` de la mesa.

4. **¡Lanza la visualización!** Usaremos `use_static_carts:=true` para crear TFs temporales y ver los carritos:

   ```
   ros2 launch burger_description display.launch.py use_static_carts:=true
   ```

✅ ¿Cómo sé si voy por buen camino? (Criterio de Éxito)

- El launch no falla con `FileNotFoundError`.
- RViz abre correctamente y **puedes ver la mesa marrón y el brazo robot**.
- Puedes usar `map` como `Fixed Frame`.
- Los TF temporales `tag_mesa -> tag_carrito1` y `tag_mesa -> tag_carrito2` son visibles en RViz.

1. El viaje del URDF hacia `robot_description`

🧠 El Concepto

Un URDF describe la estructura como texto XML, pero los nodos no trabajan directamente con el archivo en disco. El flujo real es:

1. El **Launch** lee el archivo `.urdf` como texto.
2. Ese texto se pasa como parámetro **`robot_description`**.
3. El nodo **`robot_state_publisher`** lee ese parámetro y lo combina con `/joint_states`.
4. Finalmente, ROS publica las transformaciones en `/tf` y `/tf_static`.

🔍 Exploremos el código

Abre el launch para ver este flujo: `code burger_description/launch/display.launch.py`. Busca:

- Lectura de `delivery_scene_fixed.urdf` y de los carritos.
- Cada nodo `robot_state_publisher` y su parámetro `robot_description`.

Comando rápido para encontrar estas líneas:

```
rg -n "robot_description|robot_state_publisher|car1_apriltag|car2_apriltag|delivery_scene_fixe
d" burger_description/launch/display.launch.py
```

🛠️ Ejercicio: Inspección de Nodos

1. **Lanza el sistema:** `ros2 launch burger_description display.launch.py use_static_carts:=true`
2. **Verifica publicadores separados:**

   ```
   ros2 node list
   ```

   (Deberías ver `/robot_state_publisher`, `/car1_state_publisher` y `/car2_state_publisher`).
3. **Comprueba el parámetro del publicador principal:**

   ```
   ros2 param get /robot_state_publisher robot_description | head -n 20
   ```

✅ Criterios de Éxito

- Puedes explicar por qué hay más de un `robot_state_publisher`.
- Entiendes que si agregas un URDF nuevo al proyecto, debes actualizar el launch y reconstruir el paquete.

💡 Nota Especial: El tramo temporal `use_static_carts:=true`

Es fundamental que entiendas que este modo **es solo para depuración inicial**.

- **¿Qué pregunta responde?** ¿El URDF del carrito se carga y se ve bien si recibe una pose?
- **¿Qué NO responde?** ¿La cámara detecta el tag? ¿La localización está calibrada?

En **operación real**, el responsable de ubicar los carritos no es el launch ni el URDF. Es el nodo de localización AprilTag, que publica en tiempo real: `tag_mesa -> tag_carrito*`.

Modo	Quién publica `tag_mesa -> tag_carrito*`	Resultado
Visualización (`true`)	El launch (estático)	Carritos aparecen fijos cerca de la mesa.
Esperando localización	Nadie	Carritos cargados, pero "perdidos" (no conectados al `map`).
Operación real	Nodo AprilTag	Carritos aparecen en su pose estimada real.
Simulación (Gazebo)	Plugin de Odometría	Carritos se mueven según las leyes de la física.

Regla de oro: No uses `use_static_carts:=true` al mismo tiempo que el nodo real de localización, o tendrás dos fuentes intentando publicar lo mismo, creando un árbol TF ambiguo.

> **Nota**
>
> **Fundamento de Robótica: ¿Por qué tantos nodos publicando?** En un sistema distribuido como ROS 2, no hay una "CPU central" que calcule todo. Cada `robot_state_publisher` es responsable de una pieza del rompecabezas. Esto permite que el sistema sea escalable: puedes agregar 10 carritos más simplemente lanzando 10 pequeños nodos publicadores, sin sobrecargar un solo proceso gigante.

** No modifiques el URDF para "pegar" el carrito a la mesa. Deja que el sistema de localización haga su trabajo.

2. Anatomía de un `Link`: Visual, Collision e Inertial

🧠 El Concepto

Un `link` representa una parte rígida. Conviene leerlo como tres capas distintas:

Bloque	Para qué sirve
`visual`	Geometría visible en RViz o simulador.
`collision`	Lo que usa MoveIt 2 o el motor de física para detectar choques.
`inertial`	Masa e inercia para la simulación dinámica.

Importante: El bloque `visual` no sirve para colisiones. En entornos profesionales, las colisiones suelen ser geometrías simples (cajas, cilindros) para que la planificación sea robusta y rápida.

> **Consejo**
>
> **Fundamento de Robótica: La importancia de la Inercia** Aunque para visualizar el robot no necesitamos el bloque `<inertial>`, para la **dinámica** es vital. Si quieres que el brazo Kinova se mueva de forma fluida en un simulador (o que el controlador real compense la gravedad), el sistema debe saber cuánto pesa cada link y cómo está distribuida su masa (momento de inercia). Sin esto, el brazo caería "como un peso muerto" o se movería de forma errática.

🔍 Revisión en el Proyecto

Busca estos links en `delivery_scene_fixed.urdf` usando:

```
rg -n '<link name="table_link"|<link name="staging_area"|<link name="delivery_slot_1"|<visual|<
collision|<inertial' burger_description/urdf/delivery_scene_fixed.urdf
```

Link	Qué representa	visual	collision	Observación
`table_link`	Mesa física	Malla STL y ejes	Caja simple	La colisión no incluye los ejes visuales.
`staging_area`	Zona de preparación	Caja plana	Caja simple	Es geometría de escena simplificada.
`delivery_slot_1`	Zona objetivo	Caja visual	Caja simple	Sirve como volumen lógico.
`gen3_..._link`	Brazo Kinova	Malla CAD	Malla CAD	El robot usa geometría detallada.

🛠️ Ejercicio en RViz

1. Lanza: `ros2 launch burger_description display.launch.py use_static_carts:=true`
2. En RViz, activa `RobotModel` y cambia entre vista de escena y TF.
3. Identifica la mesa y el brazo. Activa `TF` y observa que los ejes visuales de `table_link` son solo marcas de referencia, no sólidos de colisión.

✅ Criterios de Éxito

- Distingues entre geometría de escena simplificada y geometría CAD del robot.
- Puedes explicar por qué `visual` y `collision` no deben copiarse ciegamente (ej: no usar mallas complejas para colisiones si basta con una caja).

🛠️ Mini-Reto de Validación: ¿Visual o Colisión?

1. Abre `burger_description/urdf/delivery_scene_fixed.urdf`.
2. Busca el link `table_link`.
3. **Cuenta:** ¿Cuántos bloques `<visual>` tiene? ¿Y cuántos bloques `<collision>`?
4. **En RViz:** Activa la visualización de colisiones (Panel `Displays` -> `RobotModel` -> `Collision Enabled`).
5. **Pregunta:** ¿Ves las flechas de los ejes (roja, verde, azul) cuando solo activas las colisiones? ¿Por qué crees que es así?

3. `Joint`: Tipos, Ejes, Límites y `mimic`

🧠 El Concepto

Un `joint` conecta exactamente un `parent link` con un `child link`. Define cómo puede moverse el hijo respecto al padre.

Tipos principales: | Tipo | Movimiento | |---|---| | **`fixed`** | Sin movimiento. | | **`revolute`** | Rotación con límites angulares. | | **`continuous`** | Rotación sin límite (como una rueda). | | **`prismatic`** | Traslación lineal (deslizamiento). |

> **Nota**
>
> **Fundamento de Robótica: Grados de Libertad (DOF)** Cada joint que no sea `fixed` añade un **Grado de Libertad** al robot. El modelo que estás inspeccionando tiene 6 joints rotacionales del brazo, es decir **6-DOF**, como el Gen3 del laboratorio. En robótica, 6-DOF es el mínimo para alcanzar cualquier posición (XYZ) con cualquier orientación (RPY) en el espacio; un séptimo eje (la versión de 7 GDL del Gen3) añadiría **redundancia**, que permite esquivar un obstáculo sin mover la pinza del objetivo.
>
> ⚠ **El brazo del laboratorio es de 6-DOF**, así que **no** tiene esa redundancia: para cada pose alcanzable existe en general un número finito de soluciones de cinemática inversa, no un continuo. Es una diferencia real a la hora de planificar trayectorias, no un detalle de nomenclatura.

🔍 Revisión en el Proyecto

Busca los joints del Kinova en `delivery_scene_fixed.urdf`:

```
rg -n 'gen3_joint_[1-6]|mimic|limit|axis' burger_description/urdf/delivery_scene_fixed.urdf
```

Verifica en el archivo:

- `gen3_joint_2` es `revolute`, usa `axis="0 0 1"` y tiene límites `lower="-2.24"` y `upper="2.24"`.
- `gen3_joint_3` (±2.57) y `gen3_joint_5` (±2.09) también son `revolute` con tope.
- `gen3_joint_1`, `gen3_joint_4` y `gen3_joint_6` son `continuous`: giran sin límite en el modelo.
- La pinza Robotiq 2F-85 tiene **una sola articulación actuada**, `gen3_robotiq_85_left_knuckle_joint` (`revolute`, 0 a 0.8 rad). Las otras cinco (`right_knuckle`, los dos `inner_knuckle` y los dos `finger_tip`) la imitan con `mimic`; las del lado opuesto usan `multiplier="-1"` para cerrar en sentido contrario. Por eso no suman grados de libertad: el modelo completo tiene 6 GDL del brazo más 1 de la pinza.

Nota de consistencia: son los mismos límites que usa `burger_kinova_reference/config/kinova_connection.yaml` (`joint_min_rad`/`joint_max_rad`), tomados del macro oficial de 6 GDL.

🛠️ Ejercicio: ¡Mueve el robot!

1. Lanza: `ros2 launch burger_description display.launch.py use_static_carts:=true`
2. En el panel **Joint State Publisher**, mueve los deslizadores.
3. **Verifica en RViz:** Mueve `gen3_joint_2`. ¿Se observa que el slider queda limitado aproximadamente en el rango definido por el XML?
4. **Prueba la pinza:** Mueve `gen3_robotiq_85_left_knuckle_joint`. Observa cómo el dedo derecho acompaña el movimiento gracias al `mimic`.

✅ Criterios de Éxito

- Ubicas un `axis` en el XML y lo relacionas con el movimiento en RViz.
- Explicas qué problema resuelve el uso de `mimic` en un mecanismo acoplado como una pinza.

🛠️ Mini-Reto de Validación: Cazador de Límites

1. Abre el archivo `delivery_scene_fixed.urdf` y busca el joint `gen3_joint_2`.
2. **Investiga:** ¿Cuál es su valor de `upper` y `lower` limit?
3. Ahora busca `gen3_joint_1`. ¿Tiene límites numéricos? ¿Por qué?
4. **En la terminal:** Mientras el sistema corre, ejecuta:

   ```
   ros2 topic echo /joint_states
   ```

5. Mueve el robot en el GUI y observa cómo cambian los números en la terminal. ¿Los nombres de los joints coinciden con lo que viste en el XML?

4. Árbol Cinemático: map, table_link, world y Kinova

🧠 El Concepto

URDF describe un árbol. Cada `link`, salvo la raíz (`map`), debe tener un solo padre. TF2 compone estas transformaciones para saber dónde está cualquier pieza respecto a otra.

> **Importante**
>
> **Fundamento de Robótica: Cinemática Directa (Forward Kinematics)** El árbol cinemático es la base para calcular la **Cinemática Directa**. Es el proceso matemático que responde a: *"Si conozco los ángulos de todos los joints, ¿dónde está exactamente la punta del robot?"*. TF2 hace este cálculo por nosotros millones de veces por segundo sumando las transformaciones desde la base hasta la punta.

🔍 Revisión en el Proyecto

Busca las conexiones clave en `delivery_scene_fixed.urdf`:

```
rg -n 'map_to_table|table_to_world|table_to_kinova_base|tool_to_grip|table_to_tag_mesa' burger_description/urdf/delivery_scene_fixed.urdf
```

Cadena clave	Propósito
`map -> table_link`	Ubicación de la mesa en el mundo global.
`table_link -> world -> gen3_base_link`	Inserción del brazo Kinova en la mesa.
`kinova_tool_frame -> burger_grip_frame`	Punto lógico de agarre para MoveIt 2.
`table_link -> tag_mesa`	Ancla fija para la localización AprilTag.

Dato importante: El frame `world` existe como marco auxiliar porque el URDF importado del Kinova espera un padre con ese nombre.

🛠️ Ejercicio: Visualiza el árbol

1. Lanza: `ros2 launch burger_description display.launch.py use_static_carts:=true`
2. Genera el diagrama TF: `ros2 run tf2_tools view_frames`
3. Abre el `frames.pdf` resultante y ubica la cadena: `map -> table_link -> world -> gen3_base_link`.
4. Ubica también la conexión temporal: `tag_mesa -> tag_carrito1 -> car1_base_link`.

✅ Criterios de Éxito

- Explicas por qué `world` no cuelga directamente de `map`.
- Identificas que `table_to_tag_mesa` es la raíz de todo el sistema de localización de carritos.

🛠️ Mini-Reto de Validación: Rastreando el Origen

1. En una terminal, ejecuta este comando mágico para ver la relación entre dos piezas:

   ```
   ros2 run tf2_ros tf2_echo map delivery_slot_1
   ```

2. **Analiza:** El resultado te dará una traslación (At) y rotación (Rotation).
3. **Compara:** Abre el URDF y busca el joint `staging_to_delivery_slot_1`. ¿Los valores de `xyz` coinciden con lo que dice la terminal?
4. *Pista: Si no coinciden exactamente, es porque TF2 suma todas las distancias desde el 'map' hasta el hijo. ¡Intenta hacer la suma tú mismo!*

5. Ajuste de Precisión: `origin`, `xyz` y `rpy`

🧠 El Concepto

Cada `joint` tiene un `origin` que define la transformación padre-hijo:

- `xyz`: desplazamiento en metros.
- `rpy`: rotación (Roll, Pitch, Yaw) en radianes.

> **Consejo**
>
> **Fundamento de Robótica: La Regla de la Mano Derecha** En ROS (y en la mayoría de la robótica), usamos sistemas de coordenadas "derechos". - El dedo **Pulgar** apunta al eje **X** (Rojo - hacia adelante). - El dedo **Índice** apunta al eje **Y** (Verde - hacia la izquierda). - El dedo **Medio** apunta al eje **Z** (Azul - hacia arriba). ¡Memoriza esto! Te ahorrará horas de frustración cuando intentes orientar un sensor o una cámara.

🔍 Revisión: El ancla `tag_mesa`

Busca la conexión física del tag en la mesa:

```
rg -n 'table_to_tag_mesa' burger_description/urdf/delivery_scene_fixed.urdf
```

El patrón típico es:

```
<joint name="table_to_tag_mesa" type="fixed">
  <origin xyz="0.5 0.3 0.025" rpy="0 0 0" />
</joint>
```

Interpretación: 50cm en X, 30cm en Y respecto al centro de la mesa, y 2.5cm elevado sobre la superficie.

🛠️ Tu Reto: Calibración Real

1. Identifica el origen de `table_link` en RViz.
2. Mide físicamente la posición del centro del AprilTag real respecto al origen de la mesa.
3. Convierte a metros y actualiza `table_to_tag_mesa` en el URDF.
4. Si el sticker está rotado, ajusta el valor de `yaw` (el último de los tres ceros en `rpy`).

✅ Criterios de Éxito

- El frame `tag_mesa` en RViz coincide exactamente con la posición del tag físico.
- Entiendes por qué no debes mover `tag_carrito` para compensar un error que está en `tag_mesa`.

🛠️ Mini-Reto de Validación: El Desafío del Offset

1. Busca el joint `map_to_table` en el URDF de la escena.
2. **Predicción:** Si cambiamos el valor de `xyz="0.80 -1.00 0.80"` a `xyz="0.90 -1.00 0.80"`, ¿hacia dónde se moverá la mesa en RViz? (¿Adelante, atrás, izquierda o derecha?).
3. **Prueba de fuego:** Haz el cambio, compila (`colcon build`), relanza RViz y verifica si tu predicción fue correcta. ¡No olvides dejarlo como estaba después!

6. Carritos como URDFs Separados

🧠 El Concepto

Los carritos son móviles, pero su geometría interna es rígida. Separarlos de la escena fija resuelve dos responsabilidades:

1. El **URDF del carrito** define dimensiones, ruedas y offsets internos.
2. El **Nodo de Localización** define dónde está ese carrito en la mesa en cada instante.

🔍 Revisión en el Proyecto

Verifica que la escena fija no contenga los carritos:

```
rg 'car1_base_link' burger_description/urdf/delivery_scene_fixed.urdf
```

(Debería estar vacío).

Ahora inspecciona un carrito: `code burger_description/urdf/car1_apriltag.urdf`. Busca los joints que miden los offsets críticos:

- `tag_carrito1_to_car1_base` (Offset desde el sticker al centro del carro).
- `car1_base_to_delivery_tray` (Punto donde se entrega la comida).

🛠️ Ejercicio: Visualización con Carritos

1. Lanza: `ros2 launch burger_description display.launch.py use_static_carts:=true`
2. En RViz, busca los displays de tipo `RobotModel`. Deberías tener tres:
   - `RobotModel` (Escena/Kinova, topic `/robot_description`).
   - `Car1Model` (Carrito 1, topic `/car1/robot_description`).
   - `Car2Model` (Carrito 2, topic `/car2/robot_description`).
3. **Depuración:** Si ves el frame `tag_carrito1` pero no el carro, verifica que el `Description Topic` en RViz sea el correcto.

✅ Criterios de Éxito

- Visualizas los carritos temporalmente usando `use_static_carts:=true`.
- Identificas qué joint ajustar si el AprilTag físico de un carrito no está perfectamente centrado.

🛠️ Mini-Reto de Validación: El Test de Independencia

1. Abre el launch `display.launch.py` y busca el parámetro `use_static_carts`.
2. **Experimento:** Cambia el valor por defecto a `False` en el código (o lánzalo sin el parámetro en la terminal).
3. **Observa:** En RViz, ¿puedes ver la mesa y el robot? ¿Puedes ver los carritos?
4. **Pregunta:** Si los carritos no aparecen, ¿significa que el URDF del carrito está roto o que simplemente no hay nadie diciendo *dónde* está el carrito?
5. **Reflexión:** ¿Por qué esto es mejor que tener el carrito "soldado" a la mesa en el URDF fijo?

7. ¡Asegura tu trabajo! Compilación e Instalación

🧠 El Concepto

ROS 2 necesita que tus archivos estén en el espacio de instalación (`install`). Si creas un archivo nuevo o renombras uno y no reconstruyes el paquete, el sistema no lo encontrará.

🔍 Revisión: Xacro

Aunque usamos URDFs planos, ROS 2 permite usar Xacro para variables. Verifica que el sistema puede procesar tus archivos:

```
xacro burger_description/urdf/delivery_scene_fixed.urdf > /tmp/test.urdf
```

🛠️ Ejercicio: El Ciclo de Desarrollo

Cada vez que hagas un cambio importante:

1. **Compila:** `cd ~/ros2_ws && colcon build --packages-select burger_description --symlink-install`
2. **Refresca:** `source install/setup.bash`
3. **Verifica:** `ls -l install/burger_description/share/burger_description/urdf`

✅ Criterios de Éxito

- Entiendes que `src` es tu espacio de escritura y `install/share` es el espacio de lectura de ROS 2.
- El launch no falla con `FileNotFoundError` tras agregar un archivo nuevo.

8. ¡Verifica antes de lanzar! Herramientas de Validación

🧠 El Concepto

No confíes solo en RViz. Un URDF puede verse bien pero tener errores estructurales que romperán MoveIt 2 o el controlador.

🛠️ Tu Kit de Herramientas Obligatorio

Ejecuta estas pruebas para cada archivo:

1. **Validación de XML:**

   ```
   xmllint --noout burger_description/urdf/delivery_scene_fixed.urdf
   ```

2. **Validación de Árbol Cinemático:**

   ```
   check_urdf burger_description/urdf/delivery_scene_fixed.urdf
   ```

   (La raíz debe ser `map` para la escena y `tag_carrito*` para los carritos).
3. **Sintaxis del Launch:**

   ```
   python3 -m py_compile burger_description/launch/display.launch.py
   ```

✅ Criterios de Éxito

- Todas las herramientas reportan éxito (sin errores de sintaxis o de árbol).

9. MoveIt 2, Colisiones y `tool frame`

🧠 El Concepto

MoveIt 2 necesita tres cosas para planificar movimientos seguros:

1. Cadena cinemática correcta.

2. Geometría de colisión útil.
3. Un frame de herramienta (`tool frame`) bien definido.

🔍 Revisión: El punto operativo

Busca el frame semántico de agarre en `delivery_scene_fixed.urdf`:

```
rg -n 'gen3_end_effector_link|burger_grip_frame|tool_to_grip' burger_description/urdf/delivery_scene_fixed.urdf
```

Observación: `burger_grip_frame` cuelga de `kinova_tool_frame` con un offset en Z. Este es el punto que MoveIt 2 usará como objetivo para las hamburguesas.

🛠️ Ejercicio de Reflexión

Si la hamburguesa real queda 5cm más alta que el punto asumido:

- **Incorrecto:** Deformar la malla visual o mover el `map`.
- **Correcto:** Ajustar el offset de `burger_grip_frame` en el joint `tool_to_grip`.

✅ Criterios de Éxito

- Explicas la diferencia entre el link físico (`end_effector_link`) y el frame lógico de trabajo (`burger_grip_frame`).
- Entiendes por qué un objeto sin `<collision>` es invisible e intangible para MoveIt 2.

10. Integración con AprilTags

🧠 El Concepto

El sistema de visión estima dónde están los carritos respecto a la cámara, pero MoveIt 2 necesita saber dónde están respecto a la mesa. La fórmula lógica es:

```
T(tag_mesa -> tag_carrito) = inverse(T(camera -> tag_mesa)) * T(camera -> tag_carrito)
```

Después, el nodo de localización publica dinámicamente: `tag_mesa -> tag_carrito*`.

🔍 Revisión en Guía de Visión

Abre `vision_setup/LOCALIZACION_APRILTAG.md` y verifica que la cadena cinemática coincide: `tag_mesa -> tag_carrito1 -> car1_base_link -> car1_delivery_tray_frame`

🛠️ Ejercicio Final de Localización

1. **Lanza sin TFs temporales:** `ros2 launch burger_description display.launch.py`
2. **Observa:** Los carritos no aparecen. Esto confirma que el sistema está listo para recibir datos reales del nodo de visión.
3. **Verifica:** Ejecuta `ros2 run tf2_tools view_frames`. El árbol estará desconectado en el tramo `tag_mesa -> tag_carrito*` hasta que inicies el nodo de localización.

✅ Criterios de Éxito

- Explicas por qué `tag_carrito1` no debe ser un `joint fixed` dentro de la escena.
- Entiendes que si los carritos no aparecen en modo real, el primer lugar a revisar es la cámara y el nodo de detección, no el URDF.

🛠️ Mini-Reto de Validación: El Detective de Transformadas

1. Abre el archivo `vision_setup/LOCALIZACION_APRILTAG.md`.
2. Busca la fórmula matemática de la transformada.
3. **Ejercicio Mental:** Si la cámara ve que el `tag_mesa` está a 1 metro de ella (en Z) y el `tag_carrito1` está a 1.2 metros (en Z), ¿a qué distancia está el carrito de la mesa aproximadamente?
4. **En el URDF:** Mira el joint `table_to_tag_mesa`. Si movemos físicamente el AprilTag de la mesa pero no actualizamos este joint, ¿qué pasará con la ubicación de los carritos en RViz?

11. La Cámara de Muñeca: del Píxel a la Base del Brazo (puente al Lab IA 01)

🧠 El Concepto

En el Laboratorio IA 01, Gemini señala un objeto en la imagen y el pipeline lo convierte en un punto `(X, Y, Z)` en metros. Ese punto está en el **marco óptico de la cámara de color**, que se mueve con la muñeca. Para que un planificador lo use, hay que expresarlo en la **base del brazo**, y eso es exactamente lo que hace TF2 si el árbol está completo.

Dos convenciones de ejes conviven en ROS (REP 103):

Marco	X	Y	Z	Dónde lo verás
Cuerpo (body)	adelante	izquierda	arriba	`gen3_base_link`, `table_link`, `map`
Óptico	derecha en la imagen	abajo en la imagen	hacia donde mira la cámara	marcos de imagen: `*_optical_frame` y los de `kinova_vision`

El paso de uno a otro es la rotación `rpy="-1.5708 0 -1.5708"`. Búscala en las juntas `wrist_mounted_camera_*_optical_joint` del URDF.

El módulo de visión integrado del Gen3 añade tres marcos fijos, tomados del macro oficial de Kinova y colgados de `gen3_end_effector_link`:

Marco	origin xyz respecto a gen3_end_effector_link	rpy	Uso
`gen3_camera_link`	`0 0.05639 -0.00305`	`π π 0`	Raíz del módulo de visión
`gen3_camera_color_frame`	`0 0.05639 -0.00305`	`π π 0`	Imágenes de color: aquí queda el punto 3D del Lab IA 01
`gen3_camera_depth_frame`	`0.0275 0.066 -0.00305`	`π π 0`	Sensor de profundidad (RealSense D410)

`rpy = π π 0` equivale a girar 180° alrededor de Z: el eje Z del marco de la cámara coincide con el de la herramienta, que es hacia donde mira la cámara.

> **Nota**
>
> Los frames `wrist_mounted_camera_*` que cuelgan de `gen3_bracelet_link` modelan una cámara RealSense externa genérica. **No son** el módulo de visión integrado. Sirven aquí para ver la convención óptica.

🔍 Revisión en el Proyecto

```
rg -n 'gen3_camera|_module"|optical_joint' burger_description/urdf/delivery_scene_fixed.urdf
```

🛠️ Ejercicio 1: El punto de Gemini en la base, sin robot

1. Lanza `ros2 launch burger_description display.launch.py use_static_carts:=true` y deja **todos los deslizadores en 0**. En la pose cero el brazo queda vertical.
2. Consulta la cámara respecto a la base:

   ```
   ros2 run tf2_ros tf2_echo gen3_base_link gen3_camera_color_frame
   ```

   Valor esperado en la pose cero: traslación `[0.000, -0.055, 1.174]` m y rotación identidad `[0, 0, 0, 1]`.
3. Supón que el Lab IA 01 devolvió el punto `P = (0.10, 0.05, 0.60)` m en `gen3_camera_color_frame`. Calcúlalo a mano con la matriz homogénea del paso 2: `P_base = R · P + t`. Debe darte `(0.100, -0.005, 1.774)` m.
4. Compruébalo con TF2. Ejecuta este script en otra terminal con el mismo dominio:

   ```
   import rclpy
   from geometry_msgs.msg import PointStamped
   from tf2_ros import Buffer, TransformListener
   import tf2_geometry_msgs  # noqa: F401  registra la transformación de PointStamped

   rclpy.init()
   nodo = rclpy.create_node('punto_a_base')
   buffer = Buffer()
   TransformListener(buffer, nodo)
   p = PointStamped()
   p.header.frame_id = 'gen3_camera_color_frame'
   p.point.x, p.point.y, p.point.z = 0.10, 0.05, 0.60
   while rclpy.ok():
       rclpy.spin_once(nodo, timeout_sec=0.1)
       if buffer.can_transform('gen3_base_link', p.header.frame_id, rclpy.time.Time()):
           print(buffer.transform(p, 'gen3_base_link').point)
           break
   ```

5. Mueve `gen3_joint_6` y repite el paso 2. **Predice antes:** ¿cambia solo la rotación o también la traslación? Pista: la cámara no está sobre el eje de la junta, sino desplazada 5.6 cm.

🛠️ Ejercicio 2: El árbol del robot real (Ruta B del Lab IA 01)

Con el robot, los nombres son los del driver, sin prefijo `gen3_`. El árbol tiene un hueco que debes conocer antes de usar TF en el Lab IA 01:

- El driver (`kinova_connection.launch.py`) carga `kortex_description` **sin** `vision:=true`, así que publica `base_link → … → end_effector_link` y la pinza, **pero ningún marco de cámara**.
- `kinova_vision.launch.py` publica `camera_link → camera_color_frame` y `camera_link → camera_depth_frame`, pero **nadie conecta** `camera_link` **con el brazo**. El launch reducido `kinova_vision_compressed.launch.py` ni siquiera publica esos dos.

1. **Diagnostica**, con el driver y la cámara activos en la estación anfitriona:

   ```
   ros2 run tf2_ros tf2_echo base_link camera_color_frame
   ```

   Fallará con un mensaje como *"…not part of the same tree…"*: dos árboles sin conexión.

2. **Repara**: una sola estación, la anfitriona, publica el eslabón que falta con los valores oficiales. Si corre `kinova_vision.launch.py`, conecta `camera_link`:

   ```
   ros2 run tf2_ros static_transform_publisher --x 0 --y 0.05639 --z -0.00305 \
       --roll 3.141592653589793 --pitch 3.141592653589793 --yaw 0 \
       --frame-id end_effector_link --child-frame-id camera_link
   ```

   Si corre `kinova_vision_compressed.launch.py`, usa `--child-frame-id camera_color_frame` con los mismos valores. Compruébalo antes con `ros2 run tf2_ros tf2_echo camera_link camera_color_frame`.
3. **Verifica**: repite el paso 1, que ahora debe responder. El nodo del Lab IA 01 publica cada objeto como `objetivo_gemini_<i>` colgado de `camera_color_frame`, así que con el eslabón puesto obtienes el objetivo en la base del robot:

   ```
   ros2 run tf2_ros tf2_echo base_link objetivo_gemini_0
   ```

> **Advertencia**
>
> **Un solo padre por frame.** No publiques este eslabón si alguien lanza el driver con `vision:=true`: `camera_color_frame` tendría dos padres y TF2 lo rechazaría. Tampoco lo publiquen dos estaciones a la vez. Es la misma regla de unicidad que la del driver.

> **Nota**
>
> **Dos fuentes de Kinova no coinciden.** El macro oficial ubica el sensor de profundidad 27.5 mm y 9.6 mm del de color. El `kinova_vision.launch.py` usa 19.5 mm y 5 mm. El Lab IA 01 no depende de esa TF: registra la profundidad con los extrínsecos de `config_camara_kinova.json`, que se leen del robot con la Kortex API.

✅ Criterios de Éxito

- Explicas la diferencia entre un marco de cuerpo y un marco óptico, y ubicas la rotación que los relaciona.
- Obtienes con `tf2_echo` la pose de `gen3_camera_color_frame` en la pose cero y transformas un punto a `gen3_base_link`, a mano y con TF2, con el mismo resultado.
- Explicas por qué en el robot real hace falta el eslabón `end_effector_link → camera_link` y por qué debe publicarlo una sola estación.

12. Lista de Chequeo Final (Checklist)

Antes de dar por válida tu configuración, asegúrate de haber hecho este recorrido:

1. [ ] Ejecutar `xmllint` y `check_urdf` sin errores.
2. [ ] Reconstruir el paquete y verificar archivos en `install/share`.
3. [ ] Verificar en RViz que `map` es el Fixed Frame.
4. [ ] Confirmar que el brazo Kinova se mueve en el rango esperado (Joint State Publisher).
5. [ ] Verificar que la pinza Robotiq abre y cierra con `mimic`.
6. [ ] Generar `view_frames` y validar la jerarquía completa, incluidos los `gen3_camera_*` bajo `gen3_end_effector_link`.
7. [ ] Obtener `gen3_base_link → gen3_camera_color_frame` en la pose cero y transformar un punto de la cámara a la base.
8. [ ] (Con robot) Diagnosticar y reparar el eslabón `end_effector_link → camera_link` desde la estación anfitriona.

⚠️ ¡No caigas en la trampa! (Errores Comunes)

- **Olvidar reconstruir:** Cambiar el URDF en `src` y esperar que se vea en RViz sin hacer `colcon build`.
- **Meshes no encontrados:** `install/burger_description/share/burger_description/meshes` no existe. Sin esa carpeta, la mesa y objetos fijos saldrán en rojo o serán invisibles. Actualiza tu repositorio (versiones anteriores del `CMakeLists.txt` no instalaban `meshes`, y funcionaba sólo en equipos con un `install/` antiguo) y recompila. Un `install/` viejo puede ocultar el problema: ante la duda, borra `build/burger_description` e `install/burger_description` y compila de nuevo.
- **Dominio compartido:** Lanzar `display.launch.py` en el mismo dominio que otras estaciones o que una sesión con el robot (el `0` del curso). Publica `/joint_states` y `/tf` sin namespace y se mezcla con sus datos (ver aviso al inicio).
- **Doble publicación:** Usar `use_static_carts:=true` al mismo tiempo que el nodo real de AprilTags.
- **Offsets mal ubicados:** Corregir un error de la mesa moviendo el carrito. Siempre calibra de la raíz hacia las hojas.
- **Colisiones complejas:** Usar mallas CAD pesadas para detección de choques en lugar de cajas simples.
- **Confundir marcos de cámara:** Interpretar un punto del marco óptico (Z hacia delante) como si estuviera en un marco de cuerpo (Z hacia arriba), o transformarlo sin que el árbol conecte la cámara con el brazo.

🏆 Resultado Esperado (Lo que ahora sabes)

Al terminar esta guía, eres capaz de:

- Explicar cómo un URDF se convierte en el parámetro `robot_description`.
- Validar la integridad de un modelo robótico con herramientas de terminal.
- Entender por qué separamos la escena de los objetos móviles.
- Ajustar offsets físicos (`xyz`, `rpy`) para que el mundo virtual coincida con el real.
- Preparar el entorno para la planificación de movimientos con MoveIt 2.
- Llevar un punto detectado por la cámara de muñeca a la base del brazo, que es el paso que conecta la percepción del Lab IA 01 con la acción.

¡Excelente trabajo! Estás listo para el siguiente paso: **¡Control e Inteligencia con MoveIt 2!**